

Histogram and Pixel Transformation

Histogram and Pixel Transformation

Preface

Example Results

Theory

Histogram Equalization

Normalization and Linear Transformation

Lookup Table (LUT)

Code Overview

Initialize Camera and Display Module

Create Inverse Lookup Table and Set Mode

Convert to Grayscale Image

Execute Pixel Transformation

Package Results and Display

Resource Release

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Kendryte official website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

 **K230 MicroPython 镜像下载**

CI/CD 更新时间: 2026-07-28 01:29:59

选择开发板

CanMV K230 V1.0/V1.1

CanMV K230 V3.0

庐山派 K230

庐山派 Lite K230D

Yahboom K230

01Studio K230

01Studio K230 Emmc

Hiwonder K230

WonderMK K230

GT6700 K230

**Yahboom K230**
MicroPython Build for Yahboom K230
当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-28-g9b151af_nncase_v2.11.0.1mg.gz](#)

最新每日构建版本
Daily-0728
编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa
[立即下载](#)

极简 USB 烧录指南 (推荐)

⚠ 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具
下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)

☒ 有 BOOT 键的板子

☐ 无 BOOT 键的板子

Example Results

Run the example code for this section: [14.camera_equalize_normalize.py](#).

This section converts a live color image to grayscale and demonstrates histogram equalization, min-max normalization, linear scaling with absolute value, and LUT inverse mapping.

```
 

                                equalizeHist

                                normalize

                                 

                                convertScaleAbs

                                LUT
```

Theory

Histogram Equalization

A grayscale histogram counts occurrences of each gray level from 0 to 255. `equalizeHist()` remaps gray levels based on cumulative distribution to make better use of the available gray range. It is commonly used to enhance low-contrast images.

The title of this example contains "histogram," but the source code does not directly call `calcHist()` or draw a histogram. `equalizeHist()` internally uses the grayscale distribution to perform the mapping.

Normalization and Linear Transformation

`normalize(..., NORM_MINMAX)` maps the current image's minimum and maximum values to a specified range. This example maps to 0 to 255.

`convertScaleAbs()` performs approximately the following operation and outputs 8-bit absolute values:

```
dst = saturate_uint8(abs(alpha × src + beta))
```

Lookup Table (LUT)

LUT prepares output values in advance for each possible input gray level. Processing only requires looking up the table by input value, making it suitable for fixed pixel transformations like inversion, gamma correction, and piecewise mapping.

Code Overview

Initialize Camera and Display Module

```
w, h = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=w, height=h)
```

```

sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)

```

Create Inverse Lookup Table and Set Mode

```

lut = np.zeros((256, 1), dtype=np.uint8)
for i in range(256):
    lut[i, 0] = 255 - i

mode = 3

```

LUT is created outside the loop. Input 0 maps to 255, and input 255 maps to 0. The source code runs in inverse mode by default.

\$mode\$	Function	Current Parameter
0	\$equalizeHist\$	Grayscale equalization
1	\$normalize\$	0~255, \$NORM_MINMAX\$
2	\$convertScaleAbs\$	alpha=1.5, beta=-30
3	\$LUT\$	Inverse table, source default

Convert to Grayscale Image

```

gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)

```

equalizeHist() requires standard single-channel 8-bit input, so this example first converts to grayscale.

Execute Pixel Transformation

```

if mode == 0:
    out = cv2.equalizeHist(gray)
elif mode == 1:
    out = cv2.normalize(
        gray,
        alpha=0,
        beta=255,
        norm_type=cv2.NORM_MINMAX
    )
elif mode == 2:
    out = cv2.convertScaleAbs(gray, alpha=1.5, beta=-30)
else:
    out = cv2.LUT(gray, lut)

```

Equalization is a nonlinear global mapping. Min-max normalization depends on the current frame's extremes. convertScaleAbs() is a fixed linear transformation. LUT is entirely defined by the lookup table.

Package Results and Display

```
img_out = image.Image(  
    w, h, image.GRAYSCALE,  
    alloc=image.ALLOC_REF,  
    data=out  
)  
Display.show_image(img_out)
```

Resource Release

```
finally:  
    sensor.stop()  
    Display.deinit()  
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Equalization scenarios:** Low-contrast grayscale images usually improve significantly, but noise and local overexposure may also be enhanced.
- **Normalization stability:** When minimum and maximum values change each frame, image brightness may jump. For real-time projects, first calculate a stable range and then fix the mapping.
- **alpha:** Greater than 1 increases contrast, less than 1 decreases contrast.
- **beta:** Controls offset; this example uses -30. Since absolute value is also taken, the behavior in negative regions differs from ordinary brightness subtraction.
- **LUT shape and type:** The current firmware example uses a 256x1 uint8 array. Table entries must cover all 8-bit inputs.
- **Gamma transformation:** You can use $\text{output} = 255 \times (\text{input}/255)^\gamma$ to generate LUT in advance, avoiding per-pixel power operations.